

RISC-V (RV32I) Single-Cycle Core Architecture Design Report

Complete Structural Breakdown, Datapath Interconnects, and SystemVerilog RTL
Documentation

Hardware Design Team

September 15, 2026

Contents

1	System Architecture Overview	2
1.1	Architectural Block Breakdown	2
2	RV32I Instruction Set Architecture Details	3
3	RISC-V Instruction Set Architecture (ISA)	3
3.1	Instruction Formats	3
3.2	Architectural Registers	3
4	SystemVerilog Implementation	4
4.1	Package Definitions	4
4.2	Instruction Memory	6
4.3	Fetch Unit	8
4.4	Decode Unit	9
4.5	Register File	10
4.6	Arithmetic Logic Unit (ALU)	11
4.7	Data Memory	12
4.8	Branch Control	13
4.9	Control Unit	14
4.10	Top-Level Wrapper	16
4.11	Testbench	18
5	Assembly Program Example	19
5.1	Pseudocode and RISC-V Assembly	19
5.2	Machine Code and Binary Encoding	19
6	Simulation Waveform Analysis	20
6.1	Execution Trace Summary	20
6.2	Data Memory Verification	20
7	Simulation and Verification Results	21
7.1	Waveform Analysis	21
7.2	Key Signal Description and Verification Summary	21
7.3	Data Memory Result Verification	22

1 System Architecture Overview

The system implements a single-cycle 32-bit RISC-V processor executing the RV32I Base Integer Instruction Set architecture. In this design, every instruction completes within a single clock cycle.

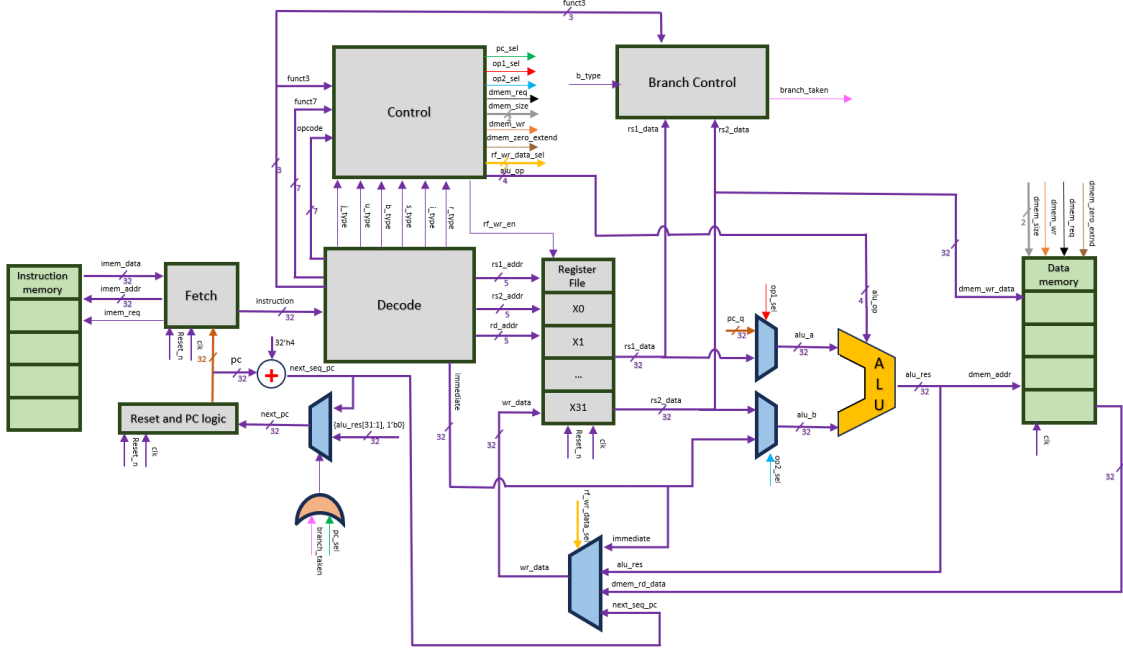


Figure 1: RISC-V Single-Cycle Core Structural Block Diagram and Interconnects

1.1 Architectural Block Breakdown

Referring to Figure 1, the hardware architecture is partitioned into the following functional units:

- **Instruction Memory (instruction_memory):** Storage containing 8-bit memory locations. It reads a full 32-bit instruction combinational word aligned to the program counter (`imem_addr`).
- **Fetch Unit (fetch) & PC Control:** Manages hardware startup and PC propagation. Calculates sequential execution paths ($PC + 4$) alongside branch/jump target addresses derived from `ALU_Res` & $\sim 1'b0$.
- **Decode Unit (decode):** Extracts register specifiers (`rs1_addr`, `rs2_addr`, `rd_addr`), opcodes, and control selectors (`funct3`, `funct7`). Sign-extends immediate bit-slices across R, I, S, B, U, and J instruction styles.
- **Register File (register_file):** Contains thirty-two 32-bit general-purpose registers (`x0-x31`). Register `x0` is hardwired to zero. Dual asynchronous read ports allow concurrent access to `rs1` and `rs2`, while writes occur synchronously on the rising clock edge.
- **Control Unit (control):** Decodes instruction opcodes and function bit patterns into driving signals for multiplexers (`op1_sel`, `op2_sel`, `rf_wr_data_sel`), memory write/read operations (`dmem_req`, `dmem_wr_en`), and arithmetic execution (`alu_op`).
- **Arithmetic Logic Unit (alu):** Performs binary arithmetic, bitwise logic operations, and comparisons based on the control signals.
- **Branch Control (branch_control):** Evaluates conditional conditions (BEQ, BNE, BLT, BGE, BLTU, BGEU) directly from source operands (`rs1_data`, `rs2_data`) to generate the `branch_taken` signal.
- **Data Memory (data_memory):** Handles byte, half-word, and word store/load operations controlled by structural flags (`dmem_size`, `dmem_zero_extend`).

2 RV32I Instruction Set Architecture Details

The base integer ISA organizes instructions into standardized 32-bit layout formats.

Table 1: RV32I Base Instruction Encoding Formats

Format	Bits 31..25	Bits 24..20	Bits 19..15	Bits 14..12	Bits 11..7	Bits 6..0
R-type	funct7	rs2	rs1	funct3	rd	opcode
I-type	imm[11:0]		rs1	funct3	rd	opcode
S-type	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B-type	imm[12—10:5]	rs2	rs1	funct3	imm[4:1—11]	opcode
U-type	imm[31:12]				rd	opcode
J-type	imm[20—10:1—11—19:12]				rd	opcode

3 RISC-V Instruction Set Architecture (ISA)

The microarchitecture implements a 32-bit RISC-V Instruction Set Architecture (ISA). The instruction formats used in this design are categorized into specific types based on their operand specifications:

- **R-Type (Register-type):** Operates on three register operands (two source registers, **rs1** and **rs2**, and one destination register, **rd**). The operation is defined by the 7-bit opcode (**op**), 3-bit function field (**funct3**), and 7-bit function field (**funct7**).
- **I-Type (Immediate-type):** Operates on two register operands (**rs1** as source, **rd** as destination) and a 12-bit immediate operand (**imm[11:0]**). Used for load instructions (e.g., **lw**) and immediate arithmetic/logic operations.
- **S-Type (Store-type):** Operates on two source register operands (**rs1** and **rs2**) and a 12-bit immediate split into two segments (**imm[11:5]** and **imm[4:0]**). Used for store instructions (e.g., **sw**).
- **B-Type (Branch-type):** Operates on two source registers (**rs1** and **rs2**) and a split 12-bit immediate to specify branch target offsets.
- **U-Type & J-Type:** Operates using a single 20-bit immediate operand for upper immediate and jump operations.

3.1 Instruction Formats

Table 2 summarizes the bit layouts for the primary 32-bit instruction formats supported by the architecture.

Table 2: RISC-V Main Instruction Formats Layout

Type	Bits 31:25	Bits 24:20	Bits 19:15	Bits 14:12	Bits 11:7	Bits 6:0
R-Type	funct7 (7b)	rs2 (5b)	rs1 (5b)	funct3 (3b)	rd (5b)	op (7b)
I-Type	imm[11:0] (12b)		rs1 (5b)	funct3 (3b)	rd (5b)	op (7b)
S-Type	imm[11:5] (7b)	rs2 (5b)	rs1 (5b)	funct3 (3b)	imm[4:0] (5b)	op (7b)
B-Type	imm[12,10:5] (7b)	rs2 (5b)	rs1 (5b)	funct3 (3b)	imm[4:1,11] (5b)	op (7b)

3.2 Architectural Registers

The architecture defines 32 general-purpose 32-bit registers (**x0** through **x31**). Table 3 presents the standard register mapping and conventions used in this design.

Table 3: RISC-V Register File Mapping

Register	ABI Name	Description / Use
x0	zero	Constant value 0
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5--x7	t0-t2	Temporary registers
x8	s0 / fp	Saved register / Frame pointer
x9	s1	Saved register
x10--x11	a0-a1	Function arguments / Return values
x12--x17	a2-a7	Function arguments
x18--x27	s2-s11	Saved registers
x28--x31	t3-t6	Temporary registers

4 SystemVerilog Implementation

4.1 Package Definitions

```

1 package risc_pkg;
2
3 typedef enum logic [6:0] {
4     OPCODE_R_TYPE    = 7'h33,
5     OPCODE_I_LOAD    = 7'h03,
6     OPCODE_I_ALU     = 7'h13,
7     OPCODE_I_JALR    = 7'h67,
8     OPCODE_S_TYPE    = 7'h23,
9     OPCODE_B_TYPE    = 7'h63,
10    OPCODE_LUI        = 7'h37,
11    OPCODE_AUIPC      = 7'h17,
12    OPCODE_JAL        = 7'h6F
13 } opcode_t;
14
15 typedef enum logic [3:0] {
16     ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
17 } alu_op_t;
18
19 typedef enum logic [1:0] {
20     BYTE      = 2'b00,
21     HALF_WORD = 2'b01,
22     WORD      = 2'b11
23 } mem_size_t;
24
25 typedef enum logic [2:0] {
26     B_BEQ = 3'h0, B_BNE = 3'h1, B_BLT = 3'h4,
27     B_BGE = 3'h5, B_BLTU = 3'h6, B_BGEU = 3'h7
28 } b_type_instr_t;
29
30 typedef enum logic [3:0] {
31     R_ADD = 4'h0, R_SUB = 4'h8, R_SLL = 4'h1,
32     R_SLT = 4'h2, R_SLTU = 4'h3, R_XOR = 4'h4,
33     R_SRL = 4'h5, R_SRA = 4'hD, R_OR = 4'h6, R_AND = 4'h7
34 } r_type_instr_t;
35
36 typedef enum logic [3:0] {
37     I_LB = 4'h0, I_LH = 4'h1, I_LW = 4'h2,
38     I_LBU = 4'h4, I_LHU = 4'h5, I_ADDI = 4'h8,
39     I_SLTI = 4'hA, I_SLTIU = 4'hB, I_XORI = 4'hC,
40     I_ORI = 4'hE, I_ANDI = 4'hF, I_SLLI = 4'h9,
41     I_SRLI_SRAI = 4'hD
42 } i_type_instr_t;
43
44 typedef enum logic [2:0] {
45     S_SB = 3'h0, S_SH = 3'h1, S_SW = 3'h2
46 } s_type_instr_t;
47
48 typedef enum logic [1:0] {
49     WB_SRC_ALU = 2'b00,
50     WB_SRC_MEM = 2'b01,
51     WB_SRC_IMM = 2'b10,

```

```

52     WB_SRC_PC = 2'b11
53 } wb_src_t;
54
55 typedef struct packed {
56     logic      mem_valid;
57     logic      mem_write;
58     mem_size_t mem_size;
59     logic      load_zero_extend;
60     logic      rf_write_enable;
61     logic      pc_src_select;
62     logic      alu_src_a_select;
63     logic      alu_src_b_select;
64     wb_src_t   wb_src;
65     alu_op_t   alu_op;
66 } control_t;
67
68 endpackage

```

Listing 1: risc_pkg.sv

4.2 Instruction Memory

```
1 module instruction_memory #(
2     parameter ADDR_WIDTH = 7,
3     parameter DATA_WIDTH = 8
4 );
5     input logic      imem_req,
6     input logic [31:0] imem_addr,
7     output logic [31:0] imem_data
8 );
9     logic [DATA_WIDTH-1:0] mem [0:(2**ADDR_WIDTH)-1];
10
11 initial begin
12     // Initialize unused memory locations to 0
13     for (int i = 112; i < (2**ADDR_WIDTH); i++) begin
14         mem[i] = 8'h00;
15     end
16     // PC 0x00: 0x00000537 -> lui x10, 0x0
17     mem[0] = 8'h37; mem[1] = 8'h05; mem[2] = 8'h00; mem[3] = 8'h00;
18
19     // PC 0x04: 0x00050513 -> addi x10, x10, 0
20     mem[4] = 8'h13; mem[5] = 8'h05; mem[6] = 8'h05; mem[7] = 8'h00;
21
22     // PC 0x08: 0x00800593 -> addi x11, x0, 8
23     mem[8] = 8'h93; mem[9] = 8'h05; mem[10] = 8'h80; mem[11] = 8'h00;
24
25     // PC 0x0C: 0x00b52023 -> sw x11, 0(x10)
26     mem[12] = 8'h23; mem[13] = 8'h20; mem[14] = 8'hb5; mem[15] = 8'h00;
27
28     // PC 0x10: 0xf0b00593 -> addi x11, x0, -21
29     mem[16] = 8'h93; mem[17] = 8'h05; mem[18] = 8'hb0; mem[19] = 8'hfe;
30
31     // PC 0x14: 0x00b52223 -> sw x11, 4(x10)
32     mem[20] = 8'h23; mem[21] = 8'h22; mem[22] = 8'hb5; mem[23] = 8'h00;
33
34     // PC 0x18: 0x00f00593 -> addi x11, x0, 15
35     mem[24] = 8'h93; mem[25] = 8'h05; mem[26] = 8'hf0; mem[27] = 8'h00;
36
37     // PC 0x1C: 0x00b52423 -> sw x11, 8(x10)
38     mem[28] = 8'h23; mem[29] = 8'h24; mem[30] = 8'hb5; mem[31] = 8'h00;
39
40     // PC 0x20: 0xffd00593 -> addi x11, x0, -3
41     mem[32] = 8'h93; mem[33] = 8'h05; mem[34] = 8'hd0; mem[35] = 8'hff;
42
43     // PC 0x24: 0x00b52623 -> sw x11, 12(x10)
44     mem[36] = 8'h23; mem[37] = 8'h26; mem[38] = 8'hb5; mem[39] = 8'h00;
45
46     // PC 0x28: 0x02a00593 -> addi x11, x0, 42
47     mem[40] = 8'h93; mem[41] = 8'h05; mem[42] = 8'ha0; mem[43] = 8'h02;
48
49     // PC 0x2C: 0x00b52823 -> sw x11, 16(x10)
50     mem[44] = 8'h23; mem[45] = 8'h28; mem[46] = 8'hb5; mem[47] = 8'h00;
51
52     // PC 0x30: 0x01100593 -> addi x11, x0, 17
53     mem[48] = 8'h93; mem[49] = 8'h05; mem[50] = 8'h10; mem[51] = 8'h01;
54
55     // PC 0x34: 0x00b52a23 -> sw x11, 20(x10)
56     mem[52] = 8'h23; mem[53] = 8'h2a; mem[54] = 8'hb5; mem[55] = 8'h00;
57
58     // PC 0x38: 0x00052583 -> lw x11, 0(x10)
59     mem[56] = 8'h83; mem[57] = 8'h25; mem[58] = 8'h05; mem[59] = 8'h00;
60
61     // PC 0x3C: 0x00100613 -> addi x12, x0, 1
62     mem[60] = 8'h13; mem[61] = 8'h06; mem[62] = 8'h10; mem[63] = 8'h00;
63
64     // PC 0x40: 0x00600693 -> addi x13, x0, 6
65     mem[64] = 8'h93; mem[65] = 8'h06; mem[66] = 8'h60; mem[67] = 8'h00;
66
67     // PC 0x44: 0x02d65263 -> bge x12, x13, 36 (offset +36)
68     mem[68] = 8'h63; mem[69] = 8'h52; mem[70] = 8'hd6; mem[71] = 8'h02;
69
70     // PC 0x48: 0x00261793 -> slli x15, x12, 2
71     mem[72] = 8'h93; mem[73] = 8'h17; mem[74] = 8'h26; mem[75] = 8'h00;
72
73     // PC 0x4C: 0x00f50833 -> add x16, x10, x15
74     mem[76] = 8'h33; mem[77] = 8'h08; mem[78] = 8'hf5; mem[79] = 8'h00;
75
```

```

76 // PC 0x50: 0x00082703 -> lw x14, 0(x16)
77 mem[80] = 8'h03; mem[81] = 8'h27; mem[82] = 8'h08; mem[83] = 8'h00;
78
79 // PC 0x54: 0x00e5c463 -> blt x11, x14, 8 (offset +8)
80 mem[84] = 8'h63; mem[85] = 8'hc4; mem[86] = 8'he5; mem[87] = 8'h00;
81
82 // PC 0x58: 0x0080006f -> jal x0, 8 (offset +8)
83 mem[88] = 8'h6f; mem[89] = 8'h00; mem[90] = 8'h80; mem[91] = 8'h00;
84
85 // PC 0x5C: 0x000705b3 -> add x11, x14, x0
86 mem[92] = 8'hb3; mem[93] = 8'h05; mem[94] = 8'h07; mem[95] = 8'h00;
87
88 // PC 0x60: 0x00160613 -> addi x12, x12, 1
89 mem[96] = 8'h13; mem[97] = 8'h06; mem[98] = 8'h16; mem[99] = 8'h00;
90
91 // PC 0x64: 0xfe1ff06f -> jal x0, -32 (loop back to PC 0x44)
92 mem[100] = 8'h6f; mem[101] = 8'hf0; mem[102] = 8'h1f; mem[103] = 8'hfe;
93
94 // PC 0x68: 0x00b52c23 -> sw x11, 24(x10)
95 mem[104] = 8'h23; mem[105] = 8'h2c; mem[106] = 8'hb5; mem[107] = 8'h00;
96
97 // PC 0x6C: 0x00000013 -> NOP (addi x0, x0, 0)
98 mem[108] = 8'h13; mem[109] = 8'h00; mem[110] = 8'h00; mem[111] = 8'h00;
99
100 end
101
102 // Use lower bits for local array indexing
103 wire [ADDR_WIDTH-1:0] base_addr = imem_addr[ADDR_WIDTH-1:0];
104
105 always_comb begin
106     if (imem_req && (base_addr + 3 < (2**ADDR_WIDTH))) begin
107         // Little-endian assembly of 32-bit instruction
108         imem_data = {
109             mem[base_addr + 3],
110             mem[base_addr + 2],
111             mem[base_addr + 1],
112             mem[base_addr]
113         };
114     end else begin
115         imem_data = 32'd0;
116     end
117 end
118
119 endmodule

```

Listing 2: instruction_memory.sv

4.3 Fetch Unit

```
1 module fetch (
2     input logic      clk,
3     input logic      reset_n,
4     input logic [31:0] pc,
5     output logic      imem_req,
6     output logic [31:0] imem_addr,
7     input logic [31:0] imem_data,
8     output logic [31:0] instruction
9 );
10     logic req_reg;
11
12     always_ff @(posedge clk or negedge reset_n) begin
13         if (!reset_n)
14             req_reg <= 1'b0;
15         else
16             req_reg <= 1'b1;
17     end
18
19     assign imem_req      = req_reg;
20     assign imem_addr     = pc;
21     assign instruction   = imem_data;
22 endmodule
```

Listing 3: fetch.sv

4.4 Decode Unit

```
1 import risc_pkg::*;
2
3 module decode(
4     input logic [31:0] instruction,
5     output logic [4:0] rs1_addr,
6     output logic [4:0] rs2_addr,
7     output logic [4:0] rd_addr,
8     output logic [6:0] opcode,
9     output logic [2:0] funct3,
10    output logic [6:0] funct7,
11    output logic      r_type,
12    output logic      i_type,
13    output logic      s_type,
14    output logic      b_type,
15    output logic      u_type,
16    output logic      j_type,
17    output logic [31:0] immediate
18 );
19     logic [31:0] imm_i_type, imm_s_type, imm_b_type, imm_u_type, imm_j_type;
20
21     assign opcode    = instruction[6:0];
22     assign rd_addr   = instruction[11:7];
23     assign funct3    = instruction[14:12];
24     assign rs1_addr  = instruction[19:15];
25     assign rs2_addr  = instruction[24:20];
26     assign funct7    = instruction[31:25];
27
28     assign imm_i_type = {{20{instruction[31]}}, instruction[31:20]};
29     assign imm_s_type = {{21{instruction[31]}}, instruction[30:25], instruction[11:7]};
30     assign imm_b_type = {{20{instruction[31]}}, instruction[7], instruction[30:25], instruction
31 [11:8], 1'b0};
32     assign imm_u_type = {instruction[31:12], 12'b0};
33     assign imm_j_type = {{12{instruction[31]}}, instruction[19:12], instruction[20],
34 instruction[30:21], 1'b0};
35
36     always_comb begin
37         r_type = 1'b0; i_type = 1'b0; s_type = 1'b0;
38         b_type = 1'b0; u_type = 1'b0; j_type = 1'b0;
39
40         case (opcode)
41             OP_CODE_R_TYPE: r_type = 1'b1;
42             OP_CODE_I_LOAD, OP_CODE_I_ALU, OP_CODE_I_JALR: i_type = 1'b1;
43             OP_CODE_S_TYPE: s_type = 1'b1;
44             OP_CODE_B_TYPE: b_type = 1'b1;
45             OP_CODE_LUI, OP_CODE_AUIPC: u_type = 1'b1;
46             OP_CODE_JAL: j_type = 1'b1;
47         endcase
48     end
49
50     assign immediate = r_type ? 32'd0 :
51                     i_type ? imm_i_type :
52                     s_type ? imm_s_type :
53                     b_type ? imm_b_type :
54                     u_type ? imm_u_type : imm_j_type;
55 endmodule
```

Listing 4: decode.sv

4.5 Register File

```
1 module register_file (
2     input logic      clk,
3     input logic      reset_n,
4     input logic [4:0] rs1_addr,
5     input logic [4:0] rs2_addr,
6     input logic [4:0] rd_addr,
7     input logic      rf_wr_en,
8     input logic [31:0] wr_data,
9     output logic [31:0] rs1_data,
10    output logic [31:0] rs2_data
11 );
12    logic [31:0] regs [0:31];
13
14    always_ff @(posedge clk or negedge reset_n) begin
15        if (!reset_n) begin
16            integer i;
17            for (i = 0; i < 32; i = i + 1)
18                regs[i] <= 32'd0;
19        end else if (rf_wr_en && (rd_addr != 5'd0)) begin
20            regs[rd_addr] <= wr_data;
21        end
22    end
23
24    assign rs1_data = regs[rs1_addr];
25    assign rs2_data = regs[rs2_addr];
26 endmodule
```

Listing 5: register_file.sv

4.6 Arithmetic Logic Unit (ALU)

```
1 import risc_pkg::*;
2
3 module alu (
4     input logic [31:0] alu_a,
5     input logic [31:0] alu_b,
6     input alu_op_t alu_op,
7     output logic [31:0] alu_res
8 );
9     logic signed [31:0] signed_a;
10    logic signed [31:0] signed_b;
11
12    assign signed_a = alu_a;
13    assign signed_b = alu_b;
14
15    always_comb begin
16        alu_res = 32'd0;
17        case (alu_op)
18            ADD : alu_res = alu_a + alu_b;
19            SUB : alu_res = alu_a - alu_b;
20            SLL : alu_res = alu_a << alu_b[4:0];
21            SRL : alu_res = alu_a >> alu_b[4:0];
22            SRA : alu_res = signed_a >>> alu_b[4:0];
23            OR  : alu_res = alu_a | alu_b;
24            AND : alu_res = alu_a & alu_b;
25            XOR : alu_res = alu_a ^ alu_b;
26            SLTU : alu_res = (alu_a < alu_b) ? 32'd1 : 32'd0;
27            SLT  : alu_res = (signed_a < signed_b) ? 32'd1 : 32'd0;
28        endcase
29    end
30 endmodule
```

Listing 6: alu.sv

4.7 Data Memory

```
1 import risc_pkg::*;
2
3 module data_memory #(
4     parameter ADDR_WIDTH = 6,
5     parameter DATA_WIDTH = 8
6 ) (
7     input logic      clk,
8     input logic      dmem_req,
9     input logic      dmem_wr_en,
10    input mem_size_t  dmem_data_size,
11    input logic [31:0] dmem_addr,
12    input logic [31:0] dmem_wr_data,
13    input logic      dmem_zero_extend,
14    output logic [31:0] dmem_rd_data
15 );
16     logic [DATA_WIDTH-1:0] mem [0:(2**ADDR_WIDTH)-1];
17
18     always_ff @(posedge clk) begin
19         if (dmem_req && dmem_wr_en) begin
20             case (dmem_data_size)
21                 BYTE:      mem[dmem_addr] <= dmem_wr_data[7:0];
22                 HALF_WORD: {mem[dmem_addr+1], mem[dmem_addr]} <= dmem_wr_data[15:0];
23                 WORD:      {mem[dmem_addr+3], mem[dmem_addr+2], mem[dmem_addr+1], mem[dmem_addr
24             ]] <= dmem_wr_data;
25             endcase
26         end
27     end
28
29     always_comb begin
30         if (dmem_req && !dmem_wr_en) begin
31             case (dmem_data_size)
32                 BYTE:      dmem_rd_data = dmem_zero_extend ? {{24{1'b0}}, mem[dmem_addr]} :
33                 {{24{mem[dmem_addr][7]}}}, mem[dmem_addr]];
34                 HALF_WORD: dmem_rd_data = dmem_zero_extend ? {{16{1'b0}}, mem[dmem_addr+1], mem
35                 [dmem_addr]} : {{16{mem[dmem_addr+1][7]}}}, mem[dmem_addr+1], mem[dmem_addr]];
36                 WORD:      dmem_rd_data = {mem[dmem_addr+3], mem[dmem_addr+2], mem[dmem_addr
37                 +1], mem[dmem_addr]};
38             endcase
39         end else begin
40             dmem_rd_data = 32'd0;
41         end
42     end
43 endmodule
```

Listing 7: data_memory.sv

4.8 Branch Control

```
1 import risc_pkg::*;
2
3 module branch_control (
4     input logic [31:0] opr_a,
5     input logic [31:0] opr_b,
6     input logic        is_b_type,
7     input logic [2:0]  funct3,
8     output logic       branch_taken
9 );
10 logic signed [31:0] a_signed, b_signed;
11 assign a_signed = opr_a;
12 assign b_signed = opr_b;
13 logic taken;
14
15 always_comb begin
16     taken = 1'b0;
17     case (funct3)
18         B_BEQ  : taken = (opr_a == opr_b);
19         B_BNE  : taken = (opr_a != opr_b);
20         B_BLT  : taken = (a_signed < b_signed);
21         B_BGE  : taken = (a_signed >= b_signed);
22         B_BLTU : taken = (opr_a < opr_b);
23         B_BGEU : taken = (opr_a >= opr_b);
24     endcase
25 end
26
27 assign branch_taken = is_b_type & taken;
28 endmodule
```

Listing 8: branch_control.sv

4.9 Control Unit

```

1  import risc_pkg::*;
2
3  module control (
4      input logic      r_type, i_type, s_type, b_type, u_type, j_type,
5      input logic [2:0] funct3,
6      input logic [6:0] funct7,
7      input logic [6:0] opcode,
8      output logic      pc_sel, op1_sel, op2_sel,
9      output alu_op_t    alu_op,
10     output wb_src_t     rf_wr_data_sel,
11     output logic        dmem_req,
12     output mem_size_t   dmem_size,
13     output logic        dmem_wr_en, dmem_zero_extend, rf_wr_en
14 );
15     logic [3:0] funct_r, opcode_i;
16     logic      funct7_bit5;
17     assign funct7_bit5 = funct7[5];
18
19     control_t ctrl_r, ctrl_i, ctrl_s, ctrl_b, ctrl_u, ctrl_j, ctrl;
20
21     assign funct_r = {funct7_bit5, funct3};
22
23     always_comb begin
24         ctrl_r = '0;
25         ctrl_r.rf_write_enable = 1'b1;
26         case (funct_r)
27             R_ADD  : ctrl_r.alu_op = ADD;
28             R_SUB  : ctrl_r.alu_op = SUB;
29             R_AND  : ctrl_r.alu_op = AND;
30             R_OR   : ctrl_r.alu_op = OR;
31             R_XOR  : ctrl_r.alu_op = XOR;
32             R_SLL  : ctrl_r.alu_op = SLL;
33             R_SRL  : ctrl_r.alu_op = SRL;
34             R_SRA  : ctrl_r.alu_op = SRA;
35             R_SLT  : ctrl_r.alu_op = SLT;
36             R_SLTU : ctrl_r.alu_op = SLTU;
37         endcase
38     end
39
40     assign opcode_i = {opcode[4], funct3};
41
42     always_comb begin
43         ctrl_i = '0;
44         ctrl_i.rf_write_enable = 1'b1;
45         ctrl_i.alu_src_b_select = 1'b1;
46         case (opcode_i)
47             I_ADDI    : ctrl_i.alu_op = ADD;
48             I_ANDI    : ctrl_i.alu_op = AND;
49             I_ORI     : ctrl_i.alu_op = OR;
50             I_XORI    : ctrl_i.alu_op = XOR;
51             I_SLLI    : ctrl_i.alu_op = SLL;
52             I_SRLI_SRAI : ctrl_i.alu_op = funct7_bit5 ? SRA : SRL;
53             I_SLTI    : ctrl_i.alu_op = SLT;
54             I_SLTIU   : ctrl_i.alu_op = SLTU;
55             I_LB      : {ctrl_i.mem_valid, ctrl_i.mem_size, ctrl_i.wb_src, ctrl_i.
load_zero_extend} = {1'b1, BYTE, WB_SRC_MEM, 1'b0};
56             I_LH      : {ctrl_i.mem_valid, ctrl_i.mem_size, ctrl_i.wb_src, ctrl_i.
load_zero_extend} = {1'b1, HALF_WORD, WB_SRC_MEM, 1'b0};
57             I_LW      : {ctrl_i.mem_valid, ctrl_i.mem_size, ctrl_i.wb_src, ctrl_i.
load_zero_extend} = {1'b1, WORD, WB_SRC_MEM, 1'b0};
58             I_LBU     : {ctrl_i.mem_valid, ctrl_i.mem_size, ctrl_i.wb_src, ctrl_i.
load_zero_extend} = {1'b1, BYTE, WB_SRC_MEM, 1'b1};
59             I_LHU     : {ctrl_i.mem_valid, ctrl_i.mem_size, ctrl_i.wb_src, ctrl_i.
load_zero_extend} = {1'b1, HALF_WORD, WB_SRC_MEM, 1'b1};
60         endcase
61
62         if (opcode == OPCODE_I_JALR) begin
63             ctrl_i.pc_src_select = 1'b1;
64             ctrl_i.wb_src       = WB_SRC_PC;
65             ctrl_i.alu_op       = ADD;
66         end
67     end
68
69     always_comb begin
70         ctrl_s = '0;

```

```

71     ctrl_s.mem_valid = 1'b1;
72     ctrl_s.mem_write = 1'b1;
73     ctrl_s.alu_src_b_select = 1'b1;
74     case (funct3)
75         S_SB : ctrl_s.mem_size = BYTE;
76         S_SH : ctrl_s.mem_size = HALF_WORD;
77         S_SW : ctrl_s.mem_size = WORD;
78     endcase
79 end
80
81 always_comb begin
82     ctrl_b = '0;
83     ctrl_b.alu_src_a_select = 1'b1;
84     ctrl_b.alu_src_b_select = 1'b1;
85     ctrl_b.alu_op = ADD;
86 end
87
88 always_comb begin
89     ctrl_u = '0;
90     ctrl_u.rf_write_enable = 1'b1;
91     case (opcode)
92         OPCODE_AUIPC: begin
93             ctrl_u.alu_src_a_select = 1'b1;
94             ctrl_u.alu_src_b_select = 1'b1;
95         end
96         OPCODE_LUI: begin
97             ctrl_u.wb_src = WB_SRC_IMM;
98         end
99     endcase
100 end
101
102 always_comb begin
103     ctrl_j = '0;
104     ctrl_j.rf_write_enable = 1'b1;
105     ctrl_j.wb_src = WB_SRC_PC;
106     ctrl_j.alu_src_a_select = 1'b1;
107     ctrl_j.alu_src_b_select = 1'b1;
108     ctrl_j.pc_src_select = 1'b1;
109 end
110
111 always_comb begin
112     ctrl = '0;
113     if (r_type) ctrl = ctrl_r;
114     else if (i_type) ctrl = ctrl_i;
115     else if (s_type) ctrl = ctrl_s;
116     else if (b_type) ctrl = ctrl_b;
117     else if (u_type) ctrl = ctrl_u;
118     else if (j_type) ctrl = ctrl_j;
119 end
120
121 assign pc_sel = ctrl.pc_src_select;
122 assign op1_sel = ctrl.alu_src_a_select;
123 assign op2_sel = ctrl.alu_src_b_select;
124 assign alu_op = ctrl.alu_op;
125 assign rf_wr_data_sel = ctrl.wb_src;
126 assign dmem_req = ctrl.mem_valid;
127 assign dmem_wr_en = ctrl.mem_write;
128 assign dmem_size = ctrl.mem_size;
129 assign dmem_zero_extend = ctrl.load_zero_extend;
130 assign rf_wr_en = ctrl.rf_write_enable;
131 endmodule

```

Listing 9: control.sv

4.10 Top-Level Wrapper

```
1 import risc_pkg::*;
2
3 module top #(
4     parameter RESET_PC = 32'h0000
5 )(
6     input logic clk,
7     input logic reset_n
8 );
9     logic          imem_req;
10    logic [31:0] imem_addr, imem_data;
11    logic          dmem_req, dmem_wr_en, dmem_zero_extend;
12    mem_size_t     dmem_size;
13    logic [31:0] dmem_addr, dmem_wr_data, dmem_rd_data;
14
15    logic [31:0] pc, next_pc, next_seq_pc;
16    logic        pc_sel, reset_seen;
17    logic [31:0] instruction, immediate, alu_a, alu_b, alu_res;
18    logic [6:0] opcode, funct7;
19    logic [2:0] funct3;
20    logic [4:0] rs1_addr, rs2_addr, rd_addr;
21    logic [31:0] rs1_data, rs2_data, wr_data;
22    logic        r_type, i_type, s_type, b_type, u_type, j_type;
23    alu_op_t     alu_op;
24    wb_src_t     rf_wr_data_sel;
25    logic        rf_wr_en, op1_sel, op2_sel, branch_taken;
26
27    always_ff @(posedge clk or negedge reset_n) begin
28        if (!reset_n) reset_seen <= 1'b0;
29        else          reset_seen <= 1'b1;
30    end
31
32    assign next_seq_pc = pc + 32'd4;
33    assign next_pc      = (branch_taken | pc_sel) ? {alu_res[31:1], 1'b0} : next_seq_pc;
34
35    always_ff @(posedge clk or negedge reset_n) begin
36        if (!reset_n)
37            pc <= RESET_PC;
38        else if (reset_seen)
39            pc <= next_pc;
40    end
41
42    instruction_memory u_instruction_memory (
43        .imem_req(imem_req), .imem_addr(imem_addr), .imem_data(imem_data)
44    );
45
46    fetch u_fetch (
47        .clk(clk), .reset_n(reset_n), .pc(pc),
48        .imem_req(imem_req), .imem_addr(imem_addr),
49        .imem_data(imem_data), .instruction(instruction)
50    );
51
52    decode u_decode (
53        .instruction(instruction), .rs1_addr(rs1_addr), .rs2_addr(rs2_addr),
54        .rd_addr(rd_addr), .opcode(opcode), .funct3(funct3), .funct7(funct7),
55        .r_type(r_type), .i_type(i_type), .s_type(s_type),
56        .b_type(b_type), .u_type(u_type), .j_type(j_type),
57        .immediate(immediate)
58    );
59
60    always_comb begin
61        case (rf_wr_data_sel)
62            WB_SRC_ALU: wr_data = alu_res;
63            WB_SRC_MEM: wr_data = dmem_rd_data;
64            WB_SRC_IMM: wr_data = immediate;
65            WB_SRC_PC : wr_data = next_seq_pc;
66        endcase
67    end
68
69    register_file u_register_file (
70        .clk(clk), .reset_n(reset_n), .rs1_addr(rs1_addr),
71        .rs2_addr(rs2_addr), .rd_addr(rd_addr), .rf_wr_en(rf_wr_en),
72        .wr_data(wr_data), .rs1_data(rs1_data), .rs2_data(rs2_data)
73    );
74
75    control u_control (
```

```

76     .r_type(r_type), .i_type(i_type), .s_type(s_type),
77     .b_type(b_type), .u_type(u_type), .j_type(j_type),
78     .funct3(funct3), .funct7(funct7), .opcode(opcode),
79     .pc_sel(pc_sel), .op1_sel(op1_sel), .op2_sel(op2_sel),
80     .alu_op(alu_op), .rf_wr_data_sel(rf_wr_data_sel),
81     .dmem_req(dmem_req), .dmem_size(dmem_size),
82     .dmem_wr_en(dmem_wr_en), .dmem_zero_extend(dmem_zero_extend),
83     .rf_wr_en(rf_wr_en)
84 );
85
86 branch_control u_branch_control (
87     .opr_a(rs1_data), .opr_b(rs2_data),
88     .is_b_type(b_type), .funct3(funct3),
89     .branch_taken(branch_taken)
90 );
91
92 assign alu_a = op1_sel ? pc          : rs1_data;
93 assign alu_b = op2_sel ? immediate : rs2_data;
94
95 alu u_alu (
96     .alu_a(alu_a), .alu_b(alu_b),
97     .alu_op(alu_op), .alu_res(alu_res)
98 );
99
100 assign dmem_addr    = alu_res;
101 assign dmem_wr_data = rs2_data;
102
103 data_memory u_data_memory (
104     .clk(clk), .dmem_req(dmem_req), .dmem_wr_en(dmem_wr_en),
105     .dmem_data_size(dmem_size), .dmem_addr(dmem_addr),
106     .dmem_wr_data(dmem_wr_data), .dmem_zero_extend(dmem_zero_extend),
107     .dmem_rd_data(dmem_rd_data)
108 );
109 endmodule

```

Listing 10: top.sv

4.11 Testbench

```
1 module test_bench();
2     logic clk;
3     logic reset_n;
4     integer cycle_count = 0;
5
6     always #5 clk = ~clk;
7
8     top #(.RESET_PC(32'h0)) dut (
9         .clk(clk),
10        .reset_n(reset_n)
11    );
12
13    always @(posedge clk) begin
14        cycle_count <= cycle_count + 1;
15        if (cycle_count == 200) begin
16            $finish;
17        end
18    end
19
20    initial begin
21        clk = 0;
22        reset_n = 0;
23        #10 reset_n = 1;
24    end
25 endmodule
```

Listing 11: test_bench.sv

5 Assembly Program Example

This section details a RISC-V assembly program that finds the maximum value in an array of six integers and stores the result back into data memory.

5.1 Pseudocode and RISC-V Assembly

The algorithm initializes an array in memory, sets initial variables for the maximum value and loop counter, and iterates through the array. Table 4 outlines the step-by-step RISC-V assembly implementation alongside its byte-addressed memory layout.

Table 4: RISC-V Assembly Code and Instruction Sequence

Address	Instruction	Description / Operation
0x0000	lui x10, 0x00000	Set base address (high bits)
0x0004	addi x10, x10, 0	x10 = base address (0x00000000)
0x0008	addi x11, x0, 8	Load 8 into x11
0x000C	sw x11, 0(x10)	A[0] = 8
0x0010	addi x11, x0, -21	Load -21 into x11
0x0014	sw x11, 4(x10)	A[1] = -21
0x0018	addi x11, x0, 15	Load 15 into x11
0x001C	sw x11, 8(x10)	A[2] = 15
0x0020	addi x11, x0, -3	Load -3 into x11
0x0024	sw x11, 12(x10)	A[3] = -3
0x0028	addi x11, x0, 42	Load 42 into x11
0x002C	sw x11, 16(x10)	A[4] = 42
0x0030	addi x11, x0, 17	Load 17 into x11
0x0034	sw x11, 20(x10)	A[5] = 17
0x0038	lw x11, 0(x10)	Initialize max = A[0] (x11)
0x003C	addi x12, x0, 1	Initialize loop index i = 1 (x12)
0x0040	addi x13, x0, 6	Set array length N = 6 (x13)
0x0044	bge x12, x13, 0x0068	Loop condition: if i >= N, jump to done
0x0048	slli x15, x12, 2	Calculate byte offset: x15 = i * 4
0x004C	add x16, x10, x15	Address calculation: x16 = &A[i]
0x0050	lw x14, 0(x16)	Load element: x14 = A[i]
0x0054	blt x11, x14, 0x005C	Branch if max < A[i] to update_max
0x0058	jal x0, 0x0060	Skip update: jump to skip_update
0x005C	add x11, x14, x0	Update max: max = A[i]
0x0060	addi x12, x12, 1	Increment loop index: i = i + 1
0x0064	jal x0, 0x0044	Jump back to loop_cond
0x0068	sw x11, 24(x10)	Store final max at A[6] (mem[24])
0x006C	addi x0, x0, 0	NOP / Halt operation

5.2 Machine Code and Binary Encoding

Table 5 provides the compiled 32-bit machine instructions represented in hexadecimal along with their raw memory byte sequence.

Table 5: Compiled Machine Code Hexadecimal Representation

Address	Instruction Hex	Memory Bytes (Little-Endian / Sequential)
0x0000	0x00000537	00 00 05 37
0x0004	0x00050513	00 05 05 13
0x0008	0x00800593	00 80 05 93
0x000C	0x00b52023	00 b5 20 23
0x0010	0xfeb00593	fe b0 05 93
0x0014	0x00b52223	00 b5 22 23
0x0018	0x00f00593	00 f0 05 93
0x001C	0x00b52423	00 b5 24 23
0x0020	0xffd00593	ff d0 05 93
0x0024	0x00b52623	00 b5 26 23
0x0028	0x02a00593	02 a0 05 93
0x002C	0x00b52823	00 b5 28 23
0x0030	0x01100593	01 10 05 93
0x0034	0x00b52a23	00 b5 2a 23
0x0038	0x00052583	00 05 25 83
0x003C	0x00100613	00 10 06 13
0x0040	0x00600693	00 60 06 93
0x0044	0x02d65263	02 d6 52 63
0x0048	0x00261793	00 26 17 93
0x004C	0x00f50833	00 f5 08 33
0x0050	0x00082703	00 08 27 03
0x0054	0x00e5c463	00 e5 c4 63
0x0058	0x0080006f	00 80 00 6f
0x005C	0x000705b3	00 07 05 b3
0x0060	0x00160613	00 16 06 13
0x0064	0xfe1ff06f	fe 1f f0 6f
0x0068	0x00b52c23	00 b5 2c 23
0x006C	0x0000013	00 00 00 13

6 Simulation Waveform Analysis

The simulation output verifies the hardware execution of the max-finder algorithm on the RISC-V RV32I microarchitecture.

6.1 Execution Trace Summary

Table 6: Simulation Waveform Signals and Observed State

Signal Name	Final Value	Observed Behavior / Description
cycle_count[31:0]	00000064	Core completed execution in 100 clock cycles.
imem_addr[31:0]	000000c8	Instruction memory fetch pointer sequence.
dmem_addr[31:0]	0000000c	Memory access address tracking data read/write locations.
dmem_wr_data[31:0]	00000000	Bus data payload during store cycles.
mem[0:63][7:0]	<i>Array Updated</i>	Data memory state showing initialization and result.

6.2 Data Memory Verification

The target array values $[8, -21, 15, -3, 42, 17]$ are successfully initialized in data memory (`mem[0:63]`), formatted in 32-bit little-endian two's complement hex:

- **A[0] (0x00):** 0x00000008 (8)
- **A[1] (0x04):** 0xffffffffeb (-21)
- **A[2] (0x08):** 0x0000000f (15)
- **A[3] (0x0c):** 0xfffffffffd (-3)

- A[4] (0x10): 0x0000002a (42)
- A[5] (0x14): 0x00000011 (17)

At the end of the loop execution, the processor correctly identifies the maximum value **42** (0x0000002a) and writes it to memory location A[6] (0x00000018 / byte offset 24).

7 Simulation and Verification Results

The functionality of the RISC-V RV32I microarchitecture executing the maximum element search algorithm was verified using behavioral simulation in Vivado.

7.1 Waveform Analysis

Figure 2 displays the simulation waveform capturing the execution trace, memory operations, and state progression of the testbench.

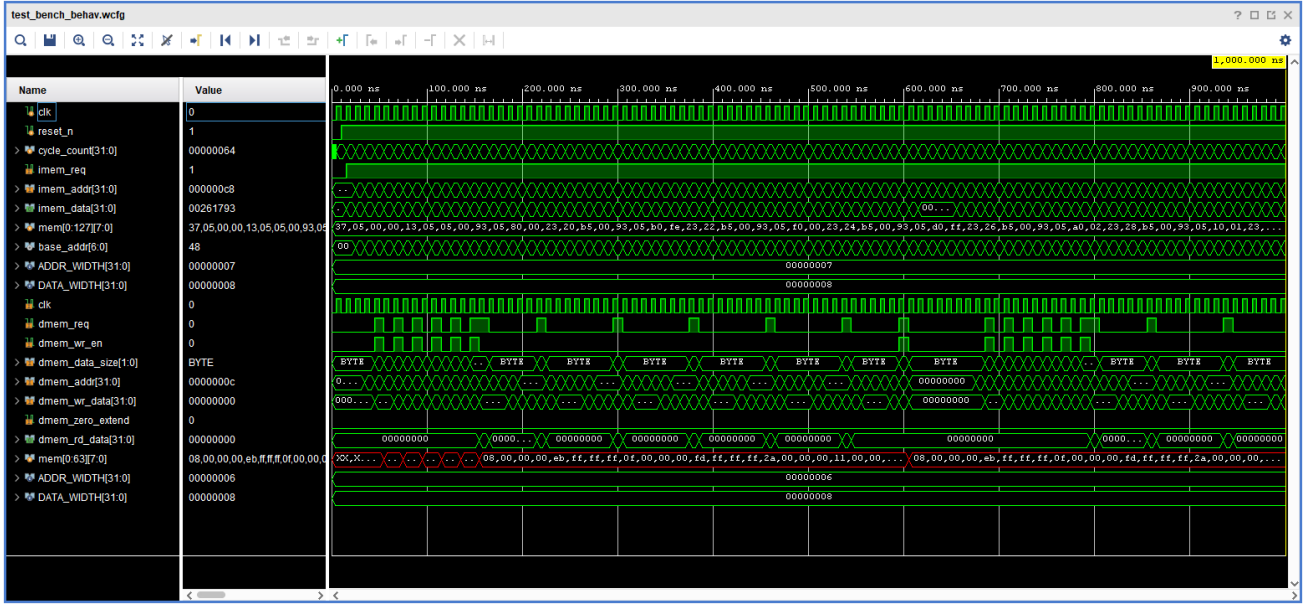


Figure 2: Behavioral Simulation Waveform for RISC-V Maximum Array Search

7.2 Key Signal Description and Verification Summary

Table 7 breaks down the key signals observed during simulation at time $t = 1,000,000$ ns, verifying correct program termination and memory modification.

Table 7: Simulation Signal Verification Summary at $t = 1000$ ns

Signal Name	Observed Value	Functional Description / Status
clk	0	System clock signal
reset_n	1	Active-low reset released; processor operating normally
cycle_count[31:0]	0x00000064	Execution completed in 100 system cycles
imem_addr[31:0]	0x000000c8	Final instruction memory fetch address
imem_data[31:0]	0x00261793	Fetches instruction (slli x15, x12, 2)
dmem_addr[31:0]	0x0000000c	Final data memory transaction address
dmem_wr_en	0	Memory write disabled at end of run
dmem_data_size[1:0]	BYTE (00)	Memory access transfer size specification
mem[0:63][7:0]	See below	Data memory layout reflecting array initialization and result

7.3 Data Memory Result Verification

The internal data memory array `mem[0:63][7:0]` reflects the array initializations followed by the execution output:

- **Array Memory Base Address (0x00):** Formatted in little-endian, memory addresses 0x00–0x17 store the elements:
 - `A[0] = 0x00000008` (08, 00, 00, 00)
 - `A[1] = 0xFFFFFEB` (eb, ff, ff, ff) (−21)
 - `A[2] = 0x0000000F` (0f, 00, 00, 00) (15)
 - `A[3] = 0xFFFFFFF` (fd, ff, ff, ff) (−3)
 - `A[4] = 0x0000002A` (2a, 00, 00, 00) (42)
 - `A[5] = 0x00000011` (11, 00, 00, 00) (17)
- **Result Address (A[6] at 0x18 / Byte Offset 24):** The maximum array value 42 (0x0000002A) is successfully calculated and stored into memory location `A[6]`, confirming expected program execution.